

Esercizio 22 Anti Taccheggio

giovedì 21 maggio 2026 12:35

Un sistema di controllo di un negozio di abbigliamento è formato da un sensore antitaccheggio posto all'uscita del negozio, da una telecamera che riprende l'uscita stessa e da un'unità di controllo basata su processore z64. La periferica SENSORE, quando rileva il passaggio di una placca antitaccheggio, invia un'interruzione all'unità di controllo. Alla ricezione di questa interruzione l'unità di controllo attiva la periferica TELECAMERA, la quale registra un filmato a bassa risoluzione e senza audio della durata di circa 10 secondi. Si supponga che i dati di registrazione di TELECAMERA siano parole a 16 bit. L'intervallo di registrazione è scandito da una periferica TIMER che viene avviata dallo z64 prima dell'attivazione della telecamera. TIMER avverte lo z64 della scadenza dell'intervallo di registrazione attraverso un'interruzione. Il filmato viene registrato in un buffer di memoria interno alla telecamera ed ha sempre la stessa dimensione: 2^{16} parole. Alla ricezione dell'interruzione da TIMER, che segna la fine del conteggio dei 10 secondi il filmato ripreso viene spostato in un buffer nella memoria centrale del sistema di controllo, attraverso un DMAC. Un nuovo filmato può sovrascrivere il precedente ma deve essere memorizzato immediatamente dopo. Si tenga presente che la periferica SENSORE non deve inviare altre interruzioni durante la registrazione di un filmato e durante la copia dei filmati in memoria principale.

Soluzione

.org 0x800

.data:

```
.equ $sensore_irq,0x0
.equ $sensore_reg,0x1
.equ $telecamera_irq,0x2
.equ $telecamera_reg,0x3
.equ $timer_irq,0x4
.equ $timer_reg,0x5
Filmato : .fill 65536,16,0 # 65k di celle a 16 bit
.equ $dmac_reg,0x6 #usato per il dmac
Buffer : .fill 65536,16,0 # 65k di celle a 16 bit
```

.text

Main :

```
Sti #abilito interruzioni
Movl $0,%al
.loop:
    Inl $sensore_reg,%al
    Cmpl $0,%al
    Jnc .loop
    Jc .attiva
.attiva:
    Movl %1,%al
    Outl %al,$sensore_irq #attivo sensore
```

.driver0: #sensore

```
Pushq %rax
Movl $0,%al
Outl %al,$sensore_irq #azzittisco richiesta
Call taccheggio
Popq %rax
Iret
```

Taccheggio:

```
Movl $1,%al
Outl %al,$time_irq #attivo timer
```

.driver2: #timer

```
Pushq %rax
Pushq %rcx
Pushq %r8 #utilizzato per ciclare sul dmac
Movl $0,%al
Outl %al,$timer_irq #azzittisco interruzione
Call incrementa
Pushq %r8
Pushq %rcx
Pushq %rax
Iret
```

Incrementa:

```

Movl %$0,%ecx
.loop1:
    Addl $1,%ecx
    Movl %ecx,%al
    Outl %al,$timer_reg
    Call f
    Cmpl $10,%ecx
    Jnc .loop1
    Jc .termina

```

F:

```

Movl $1,%al
Outl %al,$telecamera_irq #attivo telecamera

```

.termina:

```

#scrivo sul dmac
Movl $0,%r8w
.loop3: #4096 blocchi da 16
    Movl $16,%rcx
    Movl $buffer,%rsi
    Movl $dmac_reg,%dx
    Cld
    Outsb
    Addl $1,%r8w
    Cmpl $4096,%r8w
    Jnc .loop3

```

.driver1: #telecamera

```

Pushq %rax
Pushq %rcx
Movl $0,%al
Outl %al,$telecamera_irq #azzittisco interruzione
Call registra
Popq %rcx
Popq %rax
Iret

```

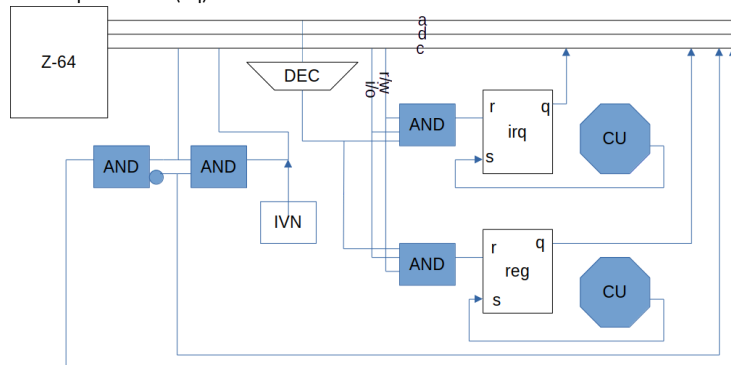
Registra:

```

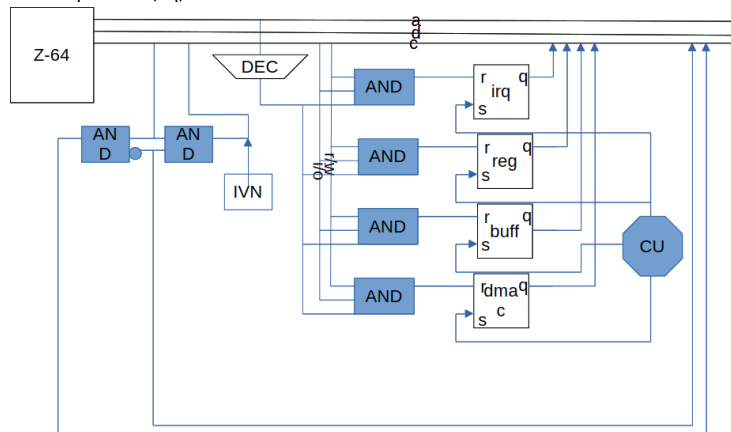
Movl $0,%ecx
Movl $0,%eax
.loop2:
    Inl $telecamera_reg,%al
    Movl %al,filmato(,%cl,2)
    Addl $16,%cl
    Cmpl $65536,%cl
    Jnc .loop2

```

Schema per sensore (irq) :



Schema per timer(irq) con dmac :



Schema per telecamera (irq) :

